

Software Development Life Cycle (SDLC) for Hospimed

This document outlines the detailed Software Development Life Cycle (SDLC) followed for the Hospimed project, a Flutter-based mobile and web application with a Supabase PostgreSQL backend. It describes the phases, activities, and deliverables for developing and maintaining the platform.

1. Requirement Analysis & Planning

1.1 Objective

To understand the business needs of managing medical and hospital equipment inventory, sales tracking, and financial analytics.

1.2 Key Activities

- **Stakeholder Interviews:** Gathered requirements regarding inventory tracking, invoice generation, customer management, and analytics (Gross Profit, Dead Stock).
- **Feasibility Study:** Evaluated the technical feasibility of using Flutter for a cross-platform frontend and Supabase (PostgreSQL) for a real-time, scalable backend.
- **Data Integration Planning:** Analyzed requirements for synchronizing data with existing ERP or accounting software (like Tally) using unique identifiers (guid, master_id).

1.3 Deliverables

- Functional and Non-Functional Requirements Document.
 - Initial Project Plan and Architecture Proposal.
-

2. System Design

2.1 Objective

To define the architecture, data models, and user interface workflows of the Hospimed platform.

2.2 Key Activities

- **Database Design:** Created robust schema definitions for `tally_companies`, `customers`, `stock_items`, `sales_invoices`, and `invoice_items`.
- **Backend Architecture:** Decided to offload heavy analytics computations (Fast/Slow moving items, Daily Profit) to the database layer using PostgreSQL Remote Procedure Calls (RPCs) to optimize frontend performance.
- **UI/UX Design:** Designed intuitive dashboards, reports screens using `fl_chart` for data visualization, and streamlined workflows for creating invoices.
- **Security Design:** Planned Row Level Security (RLS) policies in Supabase to restrict data access on a per-company basis.

2.3 Deliverables

- Database Schema Document (e.g., `supabase_schema.sql`).
 - UI Mockups and Wireframes.
 - System Architecture Diagram.
-

3. Implementation (Coding & Development)

3.1 Objective

To translate the design documents into functional code across the frontend and backend.

3.2 Key Activities

- **Backend Setup:** Initialized the Supabase project, applied the schema migrations, and wrote the PL/pgSQL functions for data aggregation (e.g., `get_daily_profit`, `get_unused_items`).
- **Frontend Development:** Developed the Flutter application integrating state management (Provider).
- **API Integration:** Connected the Flutter app to Supabase, ensuring seamless authentication and data retrieval.
- **Component Development:** Built reusable widgets like `ShimmerGridLoading`, data tables, and dynamic charts for the analytics screen.

3.3 Deliverables

- Source Code Repository (Flutter App & SQL Scripts).
 - Alpha version of the application.
-

4. Testing

4.1 Objective

To ensure the application is bug-free, meets the defined requirements, and performs efficiently under various conditions.

4.2 Key Activities

- **Unit Testing:** Executed `flutter test` to validate individual components and business logic.
- **Static Analysis:** Ran `flutter analyze` to enforce code quality, catching issues like missing annotations or edge-case nullability (e.g., preventing chart crashes when max revenue is zero).
- **Integration Testing:** Verified the end-to-end flow from creating an invoice in the app to validating the database record and ensuring it reflects correctly on the Analytics Dashboard.
- **Performance Testing:** Evaluated the RPC query response times on large datasets (e.g., thousands of invoices) to ensure dashboard charts load quickly.

4.3 Deliverables

- Test Cases and Test Logs.
 - Bug Tracking Report.
 - Beta version of the application (stable release).
-

5. Deployment

5.1 Objective

To release the application to the end-users and ensure a smooth rollout.

5.2 Key Activities

- **Backend Deployment:** Finalized production Supabase environment, enabling Row Level Security and optimizing database indexes.
- **Frontend Build:** Generated production bundles for Android (APK/AAB), iOS (IPA), and Web.
- **App Store Release:** Submitted the application to the Google Play Store and Apple App Store, handling signing certificates and review processes.
- **Environment Configuration:** Set up CI/CD pipelines (e.g., GitHub Actions) to automate future builds.

5.3 Deliverables

- Live Application accessible to users.
 - Deployment Runbooks.
-

6. Maintenance & Evaluation

6.1 Objective

To provide ongoing support, fix bugs, and add new features based on user feedback.

6.2 Key Activities

- **Monitoring:** Tracking application crashes and backend API performance using Supabase logs.
- **Bug Fixes:** Addressing user-reported issues (e.g., handling missing data gracefully in analytics).
- **Feature Enhancements:** Expanding the analytics dashboard, adding PDF invoice generation, or improving the two-way sync with Tally.
- **Database Backups:** Ensuring automated daily backups of the PostgreSQL database are running smoothly.

6.3 Deliverables

- Regular App Updates (Patch notes).
- Long-term Maintenance Strategy.